

Organización, arquitectura y plataformas de sistemas embebidos

Objetivo:

El alumno identificará las arquitecturas actuales de sistemas embebidos con plataformas comerciales.

Objetivo:

El alumno aprenderá los fundamentos de una arquitectura de computadoras...

...digo, de un sistema embebido... sí, eso...

Temario:

1. Procesador
2. Mapa de memoria del sistema
3. Controlador de interrupciones
4. Temporizadores
5. Memoria volátil y no volátil. **Direccionamientos.**
6. Bus serie universal
7. Interconexión de dispositivos
8. Secuencia de arranque en plataformas embebidas
9. Estudio de plataformas comerciales para sistemas embebidos (INTEL, ARM, etc.)

Temario:

1. Procesador
2. Mapa de memoria del sistema
3. Controlador de interrupciones
4. Temporizadores
5. Memoria volátil y no volátil. Direccinamientos
6. Bus serie universal
7. Interconexión de dispositivos
8. Secuencia de arranque en plataformas embebidas
9. Estudio de plataformas comerciales para sistemas embebidos (INTEL, ARM, etc.)

5. Direccionamientos

Direcccionamiento, modos de

- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

Direcccionamiento, modos de

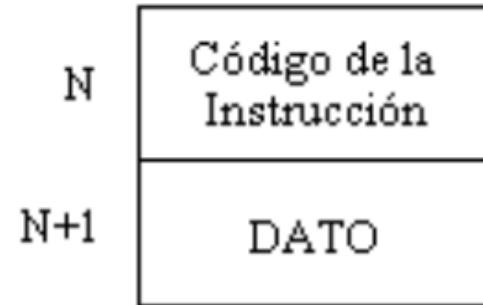
- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

No necesita operandos. El código de la instrucción es suficiente para saber el tipo de instrucción y la tarea que debe ejecutar.

Direcccionamiento, modos de

- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

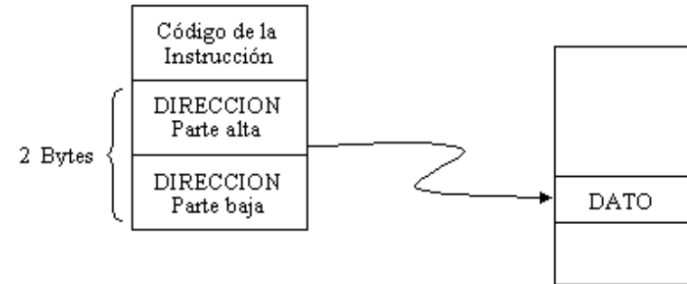
El primer byte de la instrucción corresponde a su código de operación, y el segundo byte al valor de un dato de 8 bits.



Direccionamiento, modos de

- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

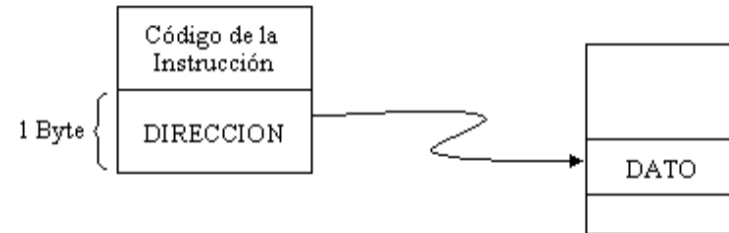
El primer byte corresponde al código de operación de la instrucción, y los dos bytes siguientes a una dirección de 16 bits que contiene el valor del operando



Direccionamiento, modos de

- Inherente
- Inmediato
- Extendido
- **Directo**
- Indexado
- Relativo

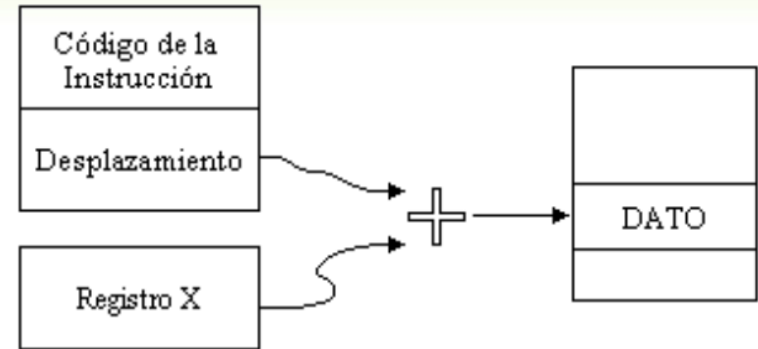
El primer byte corresponde al código de operación de la instrucción, y el segundo byte a una dirección de 8 bits que contiene el valor del operando.



Direccionamiento, modos de

- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

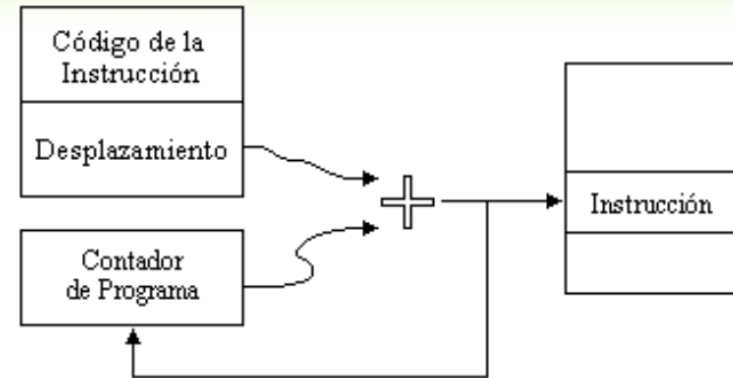
El primer byte corresponde al código de operación de la instrucción, y el segundo byte a un desplazamiento de 8 bits sin signo. La dirección del operando se calcula sumando el valor del desplazamiento más el contenido de un registro de índice.



Direccionamiento, modos de

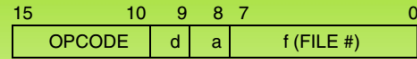
- Inherente
- Inmediato
- Extendido
- Directo
- Indexado
- Relativo

El primer byte corresponde al código de operación de la instrucción, y el segundo byte a un desplazamiento de 8 bits con signo, que se emplea para calcular la dirección de la siguiente instrucción a ejecutar.



PIC 18F4550

Byte-oriented file register operations

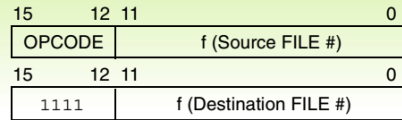


d = 0 for result destination to be WREG register
d = 1 for result destination to be file register (f)
a = 0 to force Access Bank
a = 1 for BSR to select bank
f = 8-bit file register address

Example Instruction

ADDWF MYREG, W, B

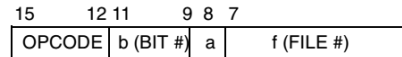
Byte to Byte move operations (2-word)



f = 12-bit file register address

MOVFF MYREG1, MYREG2

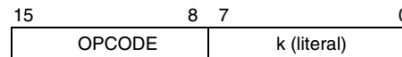
Bit-oriented file register operations



b = 3-bit position of bit in file register (f)
a = 0 to force Access Bank
a = 1 for BSR to select bank
f = 8-bit file register address

BSF MYREG, bit, B

Literal operations



k = 8-bit immediate value

MOVLW 7Fh

PIC 18F4550

TABLE 26-2: PIC18FXXXX INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected	Notes	
			MSb		LSb				
BYTE-ORIENTED OPERATIONS									
ADDWF	f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N	1, 2
ADDWFC	f, d, a	Add WREG and Carry bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N	1, 2
ANDWF	f, d, a	AND WREG with f	1	0001	01da	ffff	ffff	Z, N	1, 2
CLRF	f, a	Clear f	1	0110	101a	ffff	ffff	Z	2
COMF	f, d, a	Complement f	1	0001	11da	ffff	ffff	Z, N	1, 2
CPFSEQ	f, a	Compare f with WREG, skip =	1 (2 or 3)	0110	001a	ffff	ffff	None	4
CPFSGT	f, a	Compare f with WREG, skip >	1 (2 or 3)	0110	010a	ffff	ffff	None	4
CPFSLT	f, a	Compare f with WREG, skip <	1 (2 or 3)	0110	000a	ffff	ffff	None	1, 2
DECF	f, d, a	Decrement f	1	0000	01da	ffff	ffff	C, DC, Z, OV, N	1, 2, 3, 4
DECFSZ	f, d, a	Decrement f, Skip if 0	1 (2 or 3)	0010	11da	ffff	ffff	None	1, 2, 3, 4
DCFSNZ	f, d, a	Decrement f, Skip if Not 0	1 (2 or 3)	0100	11da	ffff	ffff	None	1, 2
INCF	f, d, a	Increment f	1	0010	10da	ffff	ffff	C, DC, Z, OV, N	1, 2, 3, 4
INCFSZ	f, d, a	Increment f, Skip if 0	1 (2 or 3)	0011	11da	ffff	ffff	None	4
INFSNZ	f, d, a	Increment f, Skip if Not 0	1 (2 or 3)	0100	10da	ffff	ffff	None	1, 2
IORWF	f, d, a	Inclusive OR WREG with f	1	0001	00da	ffff	ffff	Z, N	1, 2
MOVF	f, d, a	Move f	1	0101	00da	ffff	ffff	Z, N	1
MOVFF	f _s , f _d	Move f _s (source) to 1st word f _d (destination) 2nd word	2	1100	ffff	ffff	ffff	None	
				1111	ffff	ffff	ffff		
MOVWF	f, a	Move WREG to f	1	0110	111a	ffff	ffff	None	
MULWF	f, a	Multiply WREG with f	1	0000	001a	ffff	ffff	None	1, 2
NEGF	f, a	Negate f	1	0110	110a	ffff	ffff	C, DC, Z, OV, N	
RLCF	f, d, a	Rotate Left f through Carry	1	0011	01da	ffff	ffff	C, Z, N	1, 2
RLNCF	f, d, a	Rotate Left f (No Carry)	1	0100	01da	ffff	ffff	Z, N	
RRCF	f, d, a	Rotate Right f through Carry	1	0011	00da	ffff	ffff	C, Z, N	
RRNCF	f, d, a	Rotate Right f (No Carry)	1	0100	00da	ffff	ffff	Z, N	
SETF	f, a	Set f	1	0110	100a	ffff	ffff	None	1, 2
SUBFWB	f, d, a	Subtract f from WREG with borrow	1	0101	01da	ffff	ffff	C, DC, Z, OV, N	

Arduino ATMega

34. Instruction Set Summary

Mnemonics	Operands	Description	Operation	Flags	#Clocks
ARITHMETIC AND LOGIC INSTRUCTIONS					
ADD	Rd, Rr	Add two Registers	$Rd \leftarrow Rd + Rr$	Z, C, N, V, H	1
ADC	Rd, Rr	Add with Carry two Registers	$Rd \leftarrow Rd + Rr + C$	Z, C, N, V, H	1
ADIW	RdI, K	Add Immediate to Word	$RdH:RdL \leftarrow RdH:RdL + K$	Z, C, N, V, S	2
SUB	Rd, Rr	Subtract two Registers	$Rd \leftarrow Rd - Rr$	Z, C, N, V, H	1
SUBI	Rd, K	Subtract Constant from Register	$Rd \leftarrow Rd - K$	Z, C, N, V, H	1
SBC	Rd, Rr	Subtract with Carry two Registers	$Rd \leftarrow Rd - Rr - C$	Z, C, N, V, H	1
SBCI	Rd, K	Subtract with Carry Constant from Reg.	$Rd \leftarrow Rd - K - C$	Z, C, N, V, H	1
SBIW	RdI, K	Subtract Immediate from Word	$RdH:RdL \leftarrow RdH:RdL - K$	Z, C, N, V, S	2
AND	Rd, Rr	Logical AND Registers	$Rd \leftarrow Rd \bullet Rr$	Z, N, V	1
ANDI	Rd, K	Logical AND Register and Constant	$Rd \leftarrow Rd \bullet K$	Z, N, V	1
OR	Rd, Rr	Logical OR Registers	$Rd \leftarrow Rd \vee Rr$	Z, N, V	1
ORI	Rd, K	Logical OR Register and Constant	$Rd \leftarrow Rd \vee K$	Z, N, V	1
EOR	Rd, Rr	Exclusive OR Registers	$Rd \leftarrow Rd \oplus Rr$	Z, N, V	1
COM	Rd	One's Complement	$Rd \leftarrow 0xFF - Rd$	Z, C, N, V	1
NEG	Rd	Two's Complement	$Rd \leftarrow 0x00 - Rd$	Z, C, N, V, H	1
SBR	Rd, K	Set Bit(s) in Register	$Rd \leftarrow Rd \vee K$	Z, N, V	1
CBR	Rd, K	Clear Bit(s) in Register	$Rd \leftarrow Rd \bullet (0xFF - K)$	Z, N, V	1
INC	Rd	Increment	$Rd \leftarrow Rd + 1$	Z, N, V	1
DEC	Rd	Decrement	$Rd \leftarrow Rd - 1$	Z, N, V	1
TST	Rd	Test for Zero or Minus	$Rd \leftarrow Rd \bullet Rd$	Z, N, V	1
CLR	Rd	Clear Register	$Rd \leftarrow Rd \oplus Rd$	Z, N, V	1
SER	Rd	Set Register	$Rd \leftarrow 0xFF$	None	1
MUL	Rd, Rr	Multiply Unsigned	$R1:R0 \leftarrow Rd \times Rr$	Z, C	2
MULS	Rd, Rr	Multiply Signed	$R1:R0 \leftarrow Rd \times Rr$	Z, C	2
MULSU	Rd, Rr	Multiply Signed with Unsigned	$R1:R0 \leftarrow Rd \times Rr$	Z, C	2
FMUL	Rd, Rr	Fractional Multiply Unsigned	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z, C	2
FMULS	Rd, Rr	Fractional Multiply Signed	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z, C	2
FMULSU	Rd, Rr	Fractional Multiply Signed with Unsigned	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z, C	2

Raspberry Pi (ARM v6)

Table A4-2 summarizes the main data-processing instructions in the Thumb instruction set. The instructions are classified and described as applicable in two sections in Chapter A6 *Thumb Instruction Details*, one section for each of the following:

- INSTRUCTION (immediate) where the second operand is a modified immediate constant.
- INSTRUCTION (register) where the second operand is a register, or a register shifted by a constant.

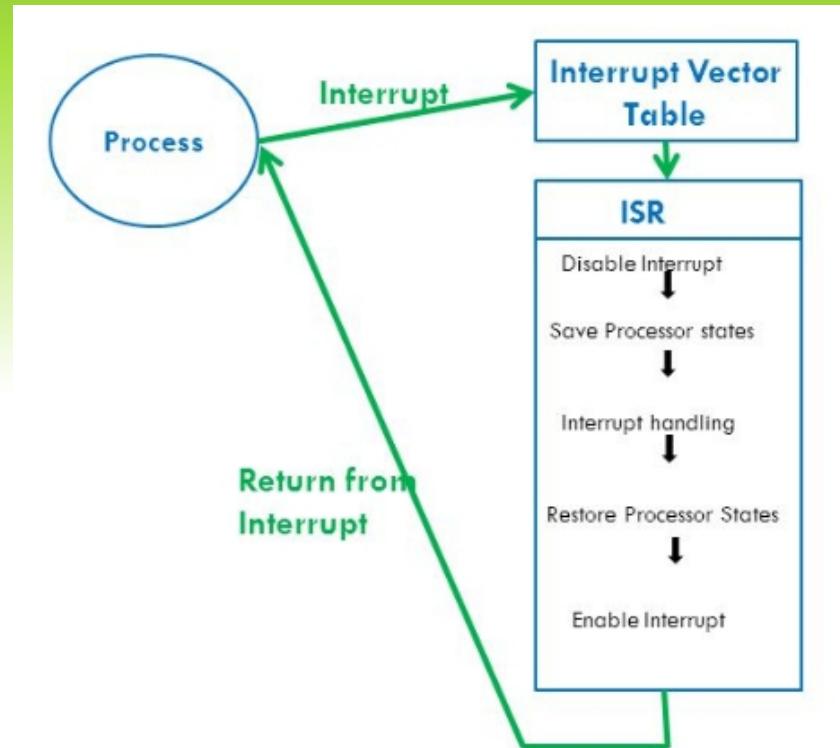
Table A4-2 Standard data-processing instructions

Mnemonic	Instruction	Notes
ADC	Add with Carry	-
ADD	Add	ARMv6-M provides register and small immediate versions only.
ADR	Form PC-relative Address	First operand is the PC. Second operand is an immediate constant.
AND	Bitwise AND	-
BIC	Bitwise Bit Clear	-
CMN	Compare Negative	Sets flags. Like ADD but with no destination register.
CMP	Compare	Sets flags. Like SUB but with no destination register.
EOR	Bitwise Exclusive OR	-

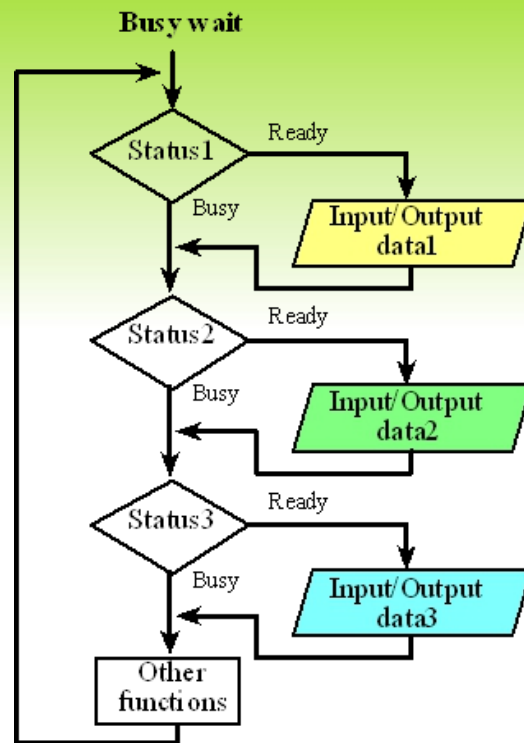
3. ICU

Controlador de Interrupciones

Interrupciones



Interrupciones



Interrupciones

- Enmascarables
- No enmascarables

Interrupciones

- Enmascarables
 - Periféricos
 - PWM
 - Timers
 - Teclado
 - Serial
 - Estados y banderas
- No enmascarables

Interrupciones

- Enmascarables
- No enmascarables
- Reset
- Overheat

